

Get Started with Intel® oneAPI Math Kernel Library

Contents

Chapter 1: Get Started with Intel® oneAPI Math Kernel Library

Get Started with Intel® oneAPI Math Kernel Library

1

The Intel® oneAPI Math Kernel Library (oneMKL) helps you achieve maximum performance with a math computing library of highly optimized and extensively parallelized routines for CPU and GPU. The library has C and Fortran interfaces for most routines on CPU and SYCL interfaces for some routines on both CPU and GPU. You can find comprehensive support for several math operations in various interfaces including:

For C and Fortran on CPU

- Linear algebra
- Fast Fourier Transforms (FFT)
- Vector math
- Direct and iterative sparse solvers
- Random number generators

For SYCL on CPU and GPU

- Linear algebra
 - BLAS
 - Selected Sparse BLAS functionality
 - Selected LAPACK(Linear Algebra PACKage) functionality
- Fast Fourier Transforms (FFT)
 - 1D, 2D, and 3D
- Random number generators
 - Selected functionality
- Selected Vector Math functionality

Refer to the [Intel® oneAPI Math Kernel Library—Data Parallel C++ Developer Reference](#) for more details.

Before You Begin

To learn about the Known Issues and get other up-to-date information, visit the [Release Notes](#) page.

For DPC++ Compiler requirements, visit the [Get Started with the Intel® oneAPI DPC++/C++ Compiler](#).

Step 1: Install Intel® oneAPI Math Kernel Library

Download [Intel® oneAPI Math Kernel Library](#) from the Intel® oneAPI Base Toolkit.

NOTE

oneMKL devel package (mkl-devel) for PIP distribution on Linux* does not provide dynamic libraries symlinks. For further details, refer to [PIP GitHub issue #5919](#).

For more information, refer to [Intel® Distribution for Python*](#).

In the case of dynamic or single dynamic library linking with the oneMKL devel package (for more information see [oneMKL Link Line Advisor](#)), you must modify the link line with the full names and versions of the oneMKL libraries.

For information about compiling and linking with the pkg-config tool, refer to the [Intel® oneAPI Math Kernel Library and pkg-config tool](#).

Here is a oneMKL link line example with the oneAPI Base Toolkit via symlinks:

Linux:

```
icx app.obj -L${MKLROOT}/lib -lmkl_intel_lp64 -lmkl_intel_thread -lmkl_core -liomp5 -lpthread -lm -ldl
```

Here is a oneMKL link line example with PIP devel package via libraries full names and versions:

Linux:

```
icx app.obj ${MKLROOT}/lib/libmkl_intel_lp64.so.2 ${MKLROOT}/lib/libmkl_intel_thread.so.2 ${MKLROOT}/lib/libmkl_core.so.2 -liomp5 -lpthread -lm -ldl
```

Step 2: Select a Function or Routine

Select a function or routine from oneMKL that is best suited for your problem. Use these resources:

Resource Link	Contents
oneMKL Developer Guide for Linux* oneMKL Developer Guide for Windows*	The Developer Guide contains detailed information on several topics including: • Compiling and linking applications • Building custom DLLs • Threading • Memory Management
oneMKL Developer Reference - C oneMKL Developer Reference - Fortran oneMKL Developer Reference - DPC++	The Developer Reference (in C, Fortran, and DPC++ formats) contains detailed descriptions of the functions and interfaces for all library domains.

Step 3: Link Your Code

Use the [oneMKL Link Line Advisor](#) to configure the link command according to your program features.

Before linking your code, consider the following limitations and additional requirements:

- Intel® oneAPI Math Kernel Library for SYCL supports the use of any interface libraries except the experimental Data Fitting domain, which supports only the mkl_intel_ilp64 interface library.
- Intel® oneAPI Math Kernel Library for SYCL supports the use of any threading libraries.

For SYCL interfaces with static linking on Linux

```
icpx -fsycl -fsycl-device-code-split=per_kernel -DMKL_ILP64 <typical user includes and linking flags and other libs> ${MKLROOT}/lib/libmkl_sycl.a -Wl,--start-group ${MKLROOT}/lib/libmkl_intel_ilp64.a ${MKLROOT}/lib/libmkl_<sequential|tbb_thread|intel_thread>.a ${MKLROOT}/lib/libmkl_core.a -Wl,--end-group -lsycl -lOpenCL -lpthread -ldl -lm
```

To build or statically link main.cpp with ilp64 interfaces and oneTBB threading, run the following command:

```
icpx -fsycl -fsycl-device-code-split=per_kernel -DMKL_ILP64 -I${MKLROOT}/include main.cpp ${MKLROOT}/lib/libmkl_sycl.a -Wl,--start-group ${MKLROOT}/lib/libmkl_intel_ilp64.a ${MKLROOT}/lib/libmkl_tbb_thread.a ${MKLROOT}/lib/libmkl_core.a -Wl,--end-group -L${TBBROOT}/lib -ltbb -lsycl -lOpenCL -lpthread -lm -ldl
```

For SYCL interfaces with dynamic linking on Linux

```
icpx -fsycl -DMKL_ILP64 <typical user includes and linking flags and other libs> -L${MKLROOT}/lib -lmkl_sycl -lmkl_intel_ilp64 -lmkl_<sequential|tbb_thread|intel_thread> -lmkl_core -lsycl -lOpenCL -lpthread -ldl -lm
```

Here is an example of building / dynamically linking main.cpp with ilp64 interfaces and oneTBB threading including all SYCL domains:

```
icpx -fsycl -DMKL_ILP64 -I${MKLROOT}/include main.cpp -L${MKLROOT}/lib -lmkl_sycl -lmkl_intel_ilp64 -lmkl_tbb_thread -lmkl_core -lsycl -lOpenCL -ltbb -lpthread -ldl -lm
```

Here is the same configuration with the BLAS SYCL domain only (note that libraries specific to the SYCL domain are aligned with oneMKL domain namespaces):

```
icpx -fsycl -DMKL_ILP64 -I${MKLROOT}/include main.cpp -L${MKLROOT}/lib -lmkl_sycl_blas -lmkl_intel_ilp64 -lmkl_tbb_thread -lmkl_core -lsycl -lOpenCL -ltbb -lpthread -ldl -lm
```

For SYCL interfaces with static linking on Windows

```
icpx -fsycl -fsycl-device-code-split=per_kernel -DMKL_ILP64 <typical user includes and linking flags and other libs> "%MKLROOT%\lib\mkl_sycl.lib mkl_intel_ilp64.lib mkl_<sequential|tbb_thread|intel_thread>.lib mkl_core.lib sycl.lib OpenCL.lib
```

Here is an example of building / statically linking main.cpp with ilp64 interfaces and oneTBB threading:

```
icpx -fsycl -fsycl-device-code-split=per_kernel -DMKL_ILP64 -I"%MKLROOT%\include" main.cpp "%MKLROOT%\lib\mkl_sycl.lib mkl_intel_ilp64.lib mkl_tbb_thread.lib mkl_core.lib sycl.lib OpenCL.lib tbb.lib
```

For SYCL interfaces with dynamic linking on Windows

```
icx -fsycl -DMKL_ILP64 <typical user includes and linking flags and other libs> "%MKLROOT%\lib\mkl_sycl_dll.lib mkl_intel_ilp64_dll.lib mkl_<sequential|tbb_thread|intel_thread>_dll.lib mkl_core_dll.lib tbb.lib sycl.lib OpenCL.lib
```

Here is an example of building / dynamically linking main.cpp with ilp64 interfaces and oneTBB threading including all SYCL domains:

```
icx -fsycl -fsycl-device-code-split=per_kernel -DMKL_ILP64 -I"%MKLROOT%\include" main.cpp "%MKLROOT%\lib\mkl_sycl_dll.lib mkl_intel_ilp64_dll.lib mkl_tbb_thread_dll.lib mkl_core_dll.lib tbb.lib sycl.lib OpenCL.lib
```

Here is the same configuration with the BLAS SYCL domain only (note that libraries specific to the SYCL domain are aligned with oneMKL domain namespaces):

```
icx -fsycl -fsycl-device-code-split=per_kernel -DMKL_ILP64 -I"%MKLROOT%\include" main.cpp "%MKLROOT%\lib\mkl_sycl_blas_dll.lib mkl_intel_ilp64_dll.lib mkl_tbb_thread_dll.lib mkl_core_dll.lib tbb.lib sycl.lib OpenCL.lib
```

For C/Fortran Interfaces with OpenMP Offload Support

Use the C/Fortran Intel® oneAPI Math Kernel Library interfaces with the OpenMP offload feature for GPU.

Add the following changes to the C/Fortran oneMKL compile/link lines to enable the OpenMP offload feature for GPU:

- Additional compile/link options: `-fiopenmp -fopenmp-targets=spir64 -mllvm -vpo-paropt-use-raw-dev-ptr -fsycl`
- Additional oneMKL library: oneMKL SYCL library

Here is an example of building / dynamically linking main.cpp on Linux with ilp64 interfaces and OpenMP threading:

```
icx -fopenmp -fopenmp-targets=spir64 -mllvm -vpo-paropt-use-raw-dev-ptr -fsycl -DMKL_ILP64 -m64 -I$(MKLROOT)/include main.cpp L$(MKLROOT)/lib -lmkl_sycl -lmkl_intel_ilp64 -lmkl_intel_thread -lmkl_core -liomp5 -lsycl -lOpenCL -lstdc++ -lpthread -lm -ldl
```

For all other supported configurations, see [Intel® oneAPI Math Kernel Library Link Line Advisor](#).

Find More

Resource	Description
oneMKL SYCL Samples in GitHub	These samples were designed to help you develop, offload, and optimize multi-architecture applications targeting CPUs and GPUs.
Tutorial: Using Intel® oneAPI Math Kernel Library for Matrix Multiplication: Tutorial - C Language Tutorial - Fortran Language	This tutorial demonstrates how you can use oneMKL to multiply matrices, measure the performance of matrix multiplication, and control threading.
Intel® oneAPI Math Kernel Library (oneMKL) Release Notes	The release notes contain information specific to the latest release of oneMKL including new and changed features. The release notes include links to principal online information resources related to the release. You can also find information on: - What's new in the release - Product contents - Obtaining technical support - License definitions
Intel® oneAPI Math Kernel Library	For support and online documentation, refer to this Intel® oneAPI Math Kernel Library (oneMKL) product page.
Intel® oneAPI Math Kernel Library Cookbook	The Intel® oneAPI Math Kernel Library contains many routines to help you solve various numerical problems, such as multiplying matrices, solving a system of equations, and performing a Fourier transform.
Notes for Intel® oneAPI Math Kernel Library Vector Statistics	This document includes an overview, a usage model, and testing results of random number generators included in VS.
Intel® oneAPI Math Kernel Library Vector Statistics Random Number Generator Performance Data	Refer to this document for performance data obtained using VS (vector statistics), RNG (random number generators) including CPE (clocks per element), BRNG (basic random number generators), generated distribution generators, and length of generated vectors.
Intel® oneAPI Math Kernel Library Vector Mathematics Performance and Accuracy Data	Vector Mathematics (VM) computes elementary functions on vector arguments. VM includes a set of highly optimized implementations of computationally expensive core mathematical functions (power, trigonometric, exponential, hyperbolic, and others) that operate on vectors.
Application Notes for Intel® oneAPI Math Kernel Library Summary Statistics	Summary Statistics is a subcomponent of the Vector Statistics domain of Intel® oneAPI Math Kernel Library. Summary Statistics provides you with functions for initial statistical analysis and offers solutions for parallel processing of multi-dimensional datasets.

Resource	Description
LAPACK Examples	This document provides code examples for oneMKL LAPACK (Linear Algebra PACKage) routines.

Notices and Disclaimers

Software and workloads used in performance tests may have been optimized for performance only on Intel microprocessors. Performance tests, such as SYSmark and MobileMark, are measured using specific computer systems, components, software, operations and functions. Any change to any of those factors may cause the results to vary. You should consult other information and performance tests to assist you in fully evaluating your contemplated purchases, including the performance of that product when combined with other products. For more complete information visit www.intel.com/benchmarks.

Intel technologies may require enabled hardware, software or service activation.

No product or component can be absolutely secure.

Your costs and results may vary.

© Intel Corporation. Intel, the Intel logo, and other Intel marks are trademarks of Intel Corporation or its subsidiaries. Other names and brands may be claimed as the property of others.

Product and Performance Information
Performance varies by use, configuration and other factors. Learn more at www.Intel.com/PerformanceIndex .
Notice revision #20201201

No license (express or implied, by estoppel or otherwise) to any intellectual property rights is granted by this document.

The products described may contain design defects or errors known as errata which may cause the product to deviate from published specifications. Current characterized errata are available on request.

Intel disclaims all express and implied warranties, including without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement, as well as any warranty arising from course of performance, course of dealing, or usage in trade.